

GameVault

Full-Stack Game Store Platform

Technical Project Report

Project Type:	Full-Stack Web Application
Frontend:	React 18 + Vite
Backend:	Node.js + Express 5
Database:	MongoDB Atlas
Deployment:	Vercel · Render · MongoDB Atlas
Live Demo:	web-project-one-lyart.vercel.app

Table of Contents

1.	Project Overview	3
2.	Features & Functionality	3
3.	System Architecture	5
4.	Technology Stack	6
5.	Database Design	7
6.	API Reference	8
7.	Security & Authentication	9
8.	User Roles & Access Control	10
9.	Frontend Structure	10
10.	Backend Structure	11
11.	Deployment & Infrastructure	12
12.	Getting Started	13
13.	Limitations & Future Work	14
14.	Conclusion	14

1. PROJECT OVERVIEW

GameVault is a full-stack web application that simulates a complete game distribution platform. It provides a functional store where customers can discover and purchase games, a developer hub where publishers can manage their titles and track performance, and an admin dashboard with full control over users, games, and platform analytics.

The project demonstrates end-to-end full-stack development: a React single-page application communicating with a RESTful Node/Express backend, secured by JWT authentication and role-based access control, with persistent data stored in MongoDB and the complete stack deployed to cloud services.

Live Demo: <https://web-project-one-lyart.vercel.app> | **API:** <https://web-project-eskq.onrender.com>

Attribute	Detail
Project Name	GameVault
Type	Full-Stack Web Application
Architecture	REST API + Single-Page Application (SPA)
Authentication	JWT (JSON Web Tokens)
Deployment	Vercel (Frontend) · Render (Backend) · MongoDB Atlas (Database)
Repository	github.com / gamevault

2. FEATURES & FUNCTIONALITY

2.1 Customer Features

The customer-facing side of GameVault provides a complete game browsing and purchasing experience, including:

- Browse the store with search, genre filter, price filter, and sort options
- Home page sections: New Releases, Top Sellers, and Free to Play
- Game detail pages with cover image, full description, screenshots, and community reviews
- Add games to a persistent cart and proceed through a mock checkout flow
- Personal game library showing all purchased titles
- Download any owned game directly from the library
- Leave a star rating and written review on any owned game
- Edit or delete own reviews at any time

2.2 Developer Features

Users with the developer role gain access to the Developer Hub, which provides:

- Publish new games with title, description, genre, price, cover image, and screenshots
- Edit or delete own published game listings
- View per-game analytics: total downloads, average rating, and individual reviews
- Track estimated revenue generated from sales of each title
- Full access to all customer features in addition to developer tools

2.3 Admin Features

Administrator accounts have unrestricted access to the platform through a dedicated dashboard:

- Platform-wide statistics: total users, total games, total orders, and total revenue
- User management: create, edit, change roles, and delete any user account
- Game management: create, edit, and delete any game across all developers
- Top-selling games list and genre breakdown chart
- All developer and customer capabilities included

2.4 General Platform Features

- JWT-based stateless authentication with configurable token expiry
- Role-based access control enforced at the API middleware layer
- Passwords hashed with bcryptjs — never stored in plaintext
- Server-side input validation with express-validator on all write endpoints
- Client-side validation for immediate user feedback
- Responsive UI designed to work on both desktop and mobile viewports
- Multipart form support for image and file uploads via multer

3. SYSTEM ARCHITECTURE

GameVault follows a standard client-server architecture with a clear separation between the frontend SPA and the backend REST API. The two communicate exclusively over HTTP using JSON payloads, with all authentication state carried in JWT tokens stored on the client.

Architecture Overview

Layer	Technology	Responsibility
Presentation	React 18 + Vite	Renders the UI, manages client state, calls the API
Routing (Client)	React Router v6	Maps URL paths to page components in the SPA
API Communication	Fetch API / api.js	Centralised HTTP calls with request/response normalisation
Application Server	Node.js + Express 5	Handles HTTP requests, routes, middleware, business logic
Authentication	jsonwebtoken	Issues and verifies signed JWT tokens on every request
Data Access	Mongoose ODM	Defines schemas and models; abstracts MongoDB queries
Database	MongoDB Atlas	Persists all users, games, orders, and reviews
File Storage	Server Filesystem	Stores uploaded cover images and screenshots (ephemeral)

Request Lifecycle

A typical authenticated API request follows this path:

1. The React frontend calls a function in `api.js`, which constructs a `fetch()` request and attaches the JWT from `localStorage` as a Bearer token in the Authorization header.
2. The Express server receives the request and passes it through the `authMiddleware`, which verifies the JWT signature and attaches the decoded user object to the request.
3. If the route requires a specific role, `roleMiddleware` checks the user's role and returns a 403 if it does not match.
4. The relevant controller function runs the business logic, queries MongoDB via the Mongoose model, and constructs a JSON response.
5. The response is returned to the client, where `api.js` normalises the data before passing it to the React component.

4. TECHNOLOGY STACK

4.1 Frontend

Technology	Version	Purpose
React	18	Component-based UI framework
Vite	5.x	Build tool and dev server with fast HMR
React Router	v6	Client-side routing and navigation
React Icons	—	Icon library (Font Awesome, Material, etc.)
Fetch API	Native	HTTP requests to the backend REST API
Context API	Built-in	Global state for auth, cart, and games

4.2 Backend

Technology	Version	Purpose
Node.js	18+	JavaScript runtime environment
Express	5.x	REST API web framework
Mongoose	8.x	MongoDB Object Document Mapper (ODM)
jsonwebtoken	—	JWT creation and verification
bcryptjs	—	Password hashing (never stored plaintext)
multer	—	Multipart form handling for file uploads
express-validator	—	Server-side input validation
cors	—	Cross-origin resource sharing configuration
dotenv	—	Environment variable management

4.3 Deployment Services

Service	Tier	Purpose
Vercel	Free	Frontend hosting with automatic GitHub deploys
Render	Free	Backend Node.js server hosting
MongoDB Atlas	Free	Managed cloud MongoDB cluster

5. DATABASE DESIGN

The database is hosted on MongoDB Atlas and uses four collections, managed through Mongoose schemas. MongoDB's document model is well-suited to this application because game data is naturally hierarchical (game → screenshots, reviews), and the schema flexibility allows rapid iteration.

5.1 User Model

Field	Type	Notes
_id	ObjectId	MongoDB auto-generated primary key
username	String	Unique, required
email	String	Unique, required, validated format
password	String	bcrypt hash — plaintext never stored
role	String	Enum: customer developer admin
createdAt	Date	Auto-set on document creation

5.2 Game Model

Field	Type	Notes
_id	ObjectId	Primary key
title	String	Required
description	String	Full game description
genre	String	Category / genre tag
price	Number	0 for free-to-play titles
coverImage	String	Filename of uploaded cover image
screenshots	[String]	Array of uploaded screenshot filenames
developer	ObjectId	Reference to User (developer role)
downloads	Number	Incremented on each download
createdAt	Date	Auto-set

5.3 Order Model

Field	Type	Notes
_id	ObjectId	Primary key
user	ObjectId	Reference to User
items	[ObjectId]	Array of Game references
total	Number	Sum of item prices at checkout
status	String	Enum: cart completed
createdAt	Date	Auto-set on document creation

5.4 Review Model

Field	Type	Notes
_id	ObjectId	Primary key
game	ObjectId	Reference to Game
user	ObjectId	Reference to User (author)
rating	Number	Integer 1–5
body	String	Written review text
createdAt	Date	Auto-set

6. API REFERENCE

All endpoints are served from the backend base URL and prefixed with `/api`. Protected routes require a valid JWT supplied as a Bearer token in the `Authorization` header.

6.1 Auth — `/api/auth`

Method	Endpoint	Auth	Description
POST	<code>/auth/register</code>	Public	Register a new user account
POST	<code>/auth/login</code>	Public	Login and receive a signed JWT
GET	<code>/auth/me</code>	Required	Return the current user's profile
PUT	<code>/auth/profile</code>	Required	Update username, email, or password

6.2 Games — `/api/games`

Method	Endpoint	Auth	Description
GET	<code>/games</code>	Public	List games (supports search, category, sort, page, limit)
GET	<code>/games/:id</code>	Public	Fetch a single game by ID
GET	<code>/games/:id/reviews</code>	Public	Fetch all reviews for a game
GET	<code>/games/:id/download</code>	Required	Download an owned game file
POST	<code>/games</code>	Developer	Publish a new game (multipart/form-data)
PUT	<code>/games/:id</code>	Developer	Update own game listing
DELETE	<code>/games/:id</code>	Developer	Delete own game listing

6.3 Orders — `/api/orders`

Method	Endpoint	Auth	Description
GET	<code>/orders/cart</code>	Required	Get the current user's cart
POST	<code>/orders/cart</code>	Required	Add a game to the cart
DELETE	<code>/orders/cart/:gameId</code>	Required	Remove a game from the cart
DELETE	<code>/orders/cart</code>	Required	Clear the entire cart
POST	<code>/orders/checkout</code>	Required	Checkout and create a completed order
GET	<code>/orders/library</code>	Required	Get all owned games (library)
GET	<code>/orders</code>	Required	Get the user's full order history

6.4 Reviews — `/api/reviews`

Method	Endpoint	Auth	Description
POST	<code>/reviews</code>	Required	Create a review for an owned game
PUT	<code>/reviews/:id</code>	Required	Update own review

Method	Endpoint	Auth	Description
DELETE	/reviews/:id	Required	Delete own review

6.5 Admin — /api/admin

Method	Endpoint	Auth	Description
GET	/admin/stats	Admin	Platform-wide stats: users, games, orders, revenue
GET	/admin/users	Admin	List all users
POST	/admin/users	Admin	Create a user
PUT	/admin/users/:id	Admin	Update a user (including role)
DELETE	/admin/users/:id	Admin	Delete a user
GET	/admin/games	Admin	List all games
POST	/admin/games	Admin	Create a game
PUT	/admin/games/:id	Admin	Update any game
DELETE	/admin/games/:id	Admin	Delete any game

7. SECURITY & AUTHENTICATION

7.1 JWT Authentication

Authentication is stateless and token-based. On successful login, the server signs a JWT containing the user's ID and role using a secret key. The token is returned to the client and stored in `localStorage`. Every subsequent request to a protected route includes the token in the `Authorization` header, where `authMiddleware` verifies it before the request reaches any controller.

7.2 Role-Based Access Control

Three roles — customer, developer, and admin — are enforced at the middleware layer. `roleMiddleware` inspects the decoded token on each request and returns HTTP 403 if the user's role does not satisfy the route's requirements. This ensures that access control cannot be bypassed by manipulating the frontend.

7.3 Password Security

All passwords are hashed using `bcryptjs` with a salt factor of 10 before being written to the database. The plaintext password is never stored or logged. On login, `bcrypt.compare()` verifies the submitted password against the stored hash.

7.4 Input Validation

Every write endpoint uses `express-validator` to validate and sanitise incoming data server-side. Client-side validation is also applied for immediate user feedback, but server-side validation is the authoritative layer — it cannot be bypassed regardless of how the API is called.

8. USER ROLES & ACCESS CONTROL

Role	Inherits From	Additional Capabilities
customer	—	Browse store, purchase games, manage cart & library, download titles, leave reviews
developer	customer	Publish games, edit/delete own listings, view per-game stats and revenue
admin	developer	Manage all users and games platform-wide, access admin dashboard and analytics

Roles are stored on the User document and included in the JWT payload. The default role for new registrations is customer. Role assignment can be changed by an admin at any time through the Admin Users management interface.

9. FRONTEND STRUCTURE

The frontend is a React 18 SPA built with Vite, organised into four main areas:

Directory / File	Purpose
src/components/layout/	Navbar, Sidebar, Footer — persistent shell components
src/components/ui/	GameCard, Icon, and other reusable UI primitives
src/context/AuthContext	Global state for the authenticated user, cart contents, and owned library
src/context/GameContext	Global games list shared across store and detail pages
src/pages/	One file per route: Home, Store, GameDetail, Cart, Checkout, Library, Login, Register
src/pages/admin/	AdminDashboard, AdminGames, AdminUsers — admin-only pages
src/services/api.js	All API calls centralised in one file with data normalisers
public/images/	Static game images and placeholder screenshots

React Router v6 handles all client-side routing. Protected routes check the auth context and redirect unauthenticated or unauthorised users before rendering. All API communication is abstracted behind api.js — components never call fetch() directly.

10. BACKEND STRUCTURE

The backend follows an MVC-style pattern. Express routes receive requests, middleware handles cross-cutting concerns, controllers execute business logic, and Mongoose models interact with MongoDB.

Directory / File	Purpose
server.js	Entry point: configures Express, middleware, and mounts route files
config/db.js	Connects to MongoDB Atlas using the MONGO_URI environment variable
models/User.js	Mongoose schema and model for user accounts
models/Game.js	Mongoose schema and model for game listings
models/Order.js	Mongoose schema and model for cart and completed orders
models/Review.js	Mongoose schema and model for game reviews
controllers/	Business logic for auth, games, orders, reviews, and admin operations
routes/	Express Router instances that map HTTP methods to controller functions
middleware/authMiddleware	Verifies JWT on every protected request; attaches user to req.user
middleware/roleMiddleware	Checks req.user.role against the required role for the route
middleware/errorMiddleware	Global error handler returning structured JSON error responses
scripts/seed.js	Database seeder populating sample games, users, and orders
uploads/	Stores uploaded cover images and screenshot files via multer

11. DEPLOYMENT & INFRASTRUCTURE

11.1 Deployment Overview

Component	Service	URL / Notes
Frontend	Vercel	web-project-one-lyart.vercel.app — auto-deploys from main branch
Backend	Render	web-project-eskq.onrender.com — Node.js web service
Database	MongoDB Atlas	Free-tier shared cluster — connection via MONGO_URI env variable

11.2 Environment Variables

Backend (gamevault-backend/.env):

Variable	Description	Example
PORT	Port the server listens on	5000
MONGO_URI	MongoDB Atlas connection string	mongodb+srv://user:pass@cluster/gamevault
JWT_SECRET	Secret key for signing JWTs	a_long_random_string
JWT_EXPIRE	Token expiry duration	7d
CLIENT_URL	Allowed frontend origin (CORS)	https://your-app.vercel.app

Frontend (gamevault-frontend/.env):

Variable	Description	Example
VITE_API_URL	Base URL of the backend REST API	https://your-backend.onrender.com/api

11.3 Known Deployment Limitations

- Render free tier spins down after 15 minutes of inactivity. The first request after a sleep period may take 30–60 seconds while the server wakes up.
- File uploads (cover images, screenshots) are stored on the server filesystem. On Render's free tier this storage is ephemeral and will not persist across restarts. A production deployment should migrate uploads to a service such as Cloudinary or AWS S3.
- After deploying the frontend to Vercel, CLIENT_URL in the Render environment must be updated to the Vercel URL and the backend must be redeployed for CORS to function correctly.

12. GETTING STARTED (LOCAL DEVELOPMENT)

Prerequisites:

- Node.js 18 or higher
- A MongoDB Atlas account (free tier is sufficient)
- Git

Step 1 — Clone the repository

```
git clone  
cd
```

Step 2 — Install backend dependencies

```
cd gamevault-backend  
npm install
```

Step 3 — Configure backend environment

```
# Create gamevault-backend/.env  
  
PORT=5000  
  
MONGO_URI=mongodb+srv://...  
  
JWT_SECRET=your_secret_here  
  
JWT_EXPIRE=7d  
  
CLIENT_URL=http://localhost:5173
```

Step 4 — Seed and start the backend

```
npm run seed # optional – loads sample data  
npm run dev # starts on http://localhost:5000
```

Step 5 — Install frontend dependencies

```
cd ../gamevault-frontend  
npm install
```

Step 6 — Configure frontend environment

```
# Create gamevault-frontend/.env  
  
VITE_API_URL=http://localhost:5000/api
```

Step 7 — Start the frontend

```
npm run dev # starts on http://localhost:5173
```

13. LIMITATIONS & FUTURE WORK

13.1 Current Limitations

- Payment processing is mocked — no real payment gateway (e.g. Stripe) is integrated.
- Game downloads serve placeholder files rather than actual game executables.
- File uploads are stored on the ephemeral server filesystem, making them unsuitable for production without migrating to cloud storage.
- Email verification and password reset flows are not implemented.
- No rate limiting is applied to API endpoints, which could allow abuse in production.
- Render free-tier cold starts introduce latency on the first request after inactivity.

13.2 Potential Improvements

- Integrate Stripe or a similar payment gateway for real transaction processing.
- Migrate file storage to Cloudinary or AWS S3 for persistent, scalable image hosting.
- Add email verification on registration and a password reset flow.
- Implement API rate limiting and request throttling to prevent abuse.
- Add pagination and infinite scroll on the store page for large game catalogues.
- Introduce a wishlist feature for customers to save games they intend to purchase.
- Build a real-time notifications system for reviews and sales using WebSockets.
- Add automated testing (Jest, Supertest) for the backend API and React Testing Library for the frontend.
- Implement refresh token rotation to improve JWT security.

14. CONCLUSION

GameVault is a complete, end-to-end full-stack web application demonstrating the development of a real-world platform from concept to deployed product. The project covers the full spectrum of modern web development: a responsive React SPA, a RESTful Node/Express API, JWT authentication with role-based access control, MongoDB persistence, and cloud deployment across Vercel, Render, and MongoDB Atlas.

The three-tier role system — customer, developer, and admin — provides a realistic model of how platform access is managed in production applications. The clean separation between frontend and backend, the centralised API service layer, and the MVC-structured backend all reflect industry-standard practices that translate directly to professional development environments.

While several limitations exist in the current version — most notably mock payments and ephemeral file storage — the architecture is designed to accommodate these improvements incrementally. The codebase provides a solid foundation for adding production-grade features such as real payment processing, persistent cloud storage, and automated testing.

Live Demo: <https://web-project-one-lyart.vercel.app> | **API:** <https://web-project-eskq.onrender.com>